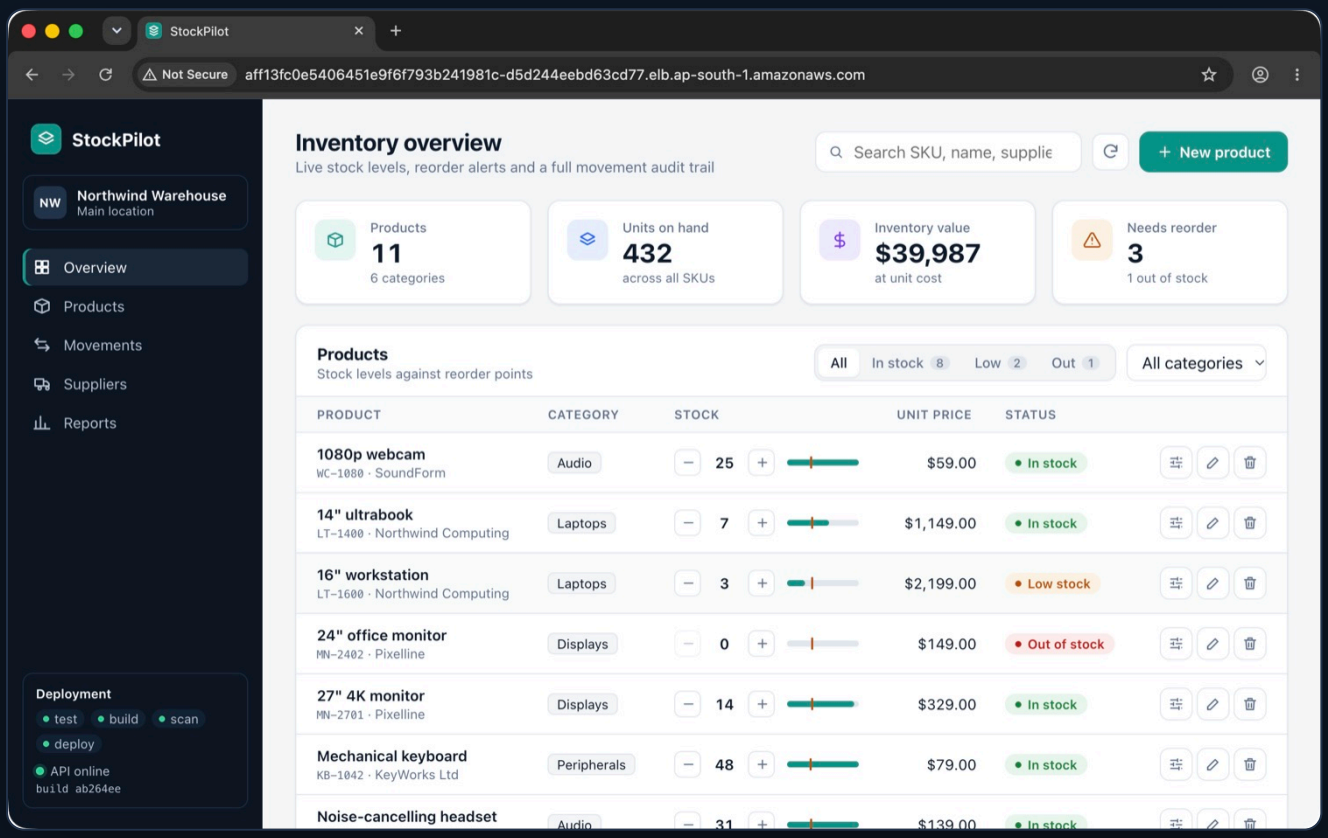
DEVOPS FINAL CAPSTONE · PROJECT REPORT

StockPilot

An inventory management application, taken from a developer laptop to an autoscaled, monitored Kubernetes cluster on AWS through a fully automated pipeline.



Author
Shambhu Yadav

Repository
github.com/paradoxicsys/stockpilot

Cloud
AWS EKS · ap-south-1

Date
October 2026

01 Project overview

Summary

StockPilot helps a small warehouse or shop keep track of what it holds. Each product has a SKU, a supplier, a unit price and a **reorder level**, so its status changes automatically between *in stock*, *low stock* and *out of stock*. Every change to a quantity is recorded as a **stock movement** (restock, sale, return, write-off or correction), giving a complete audit trail.

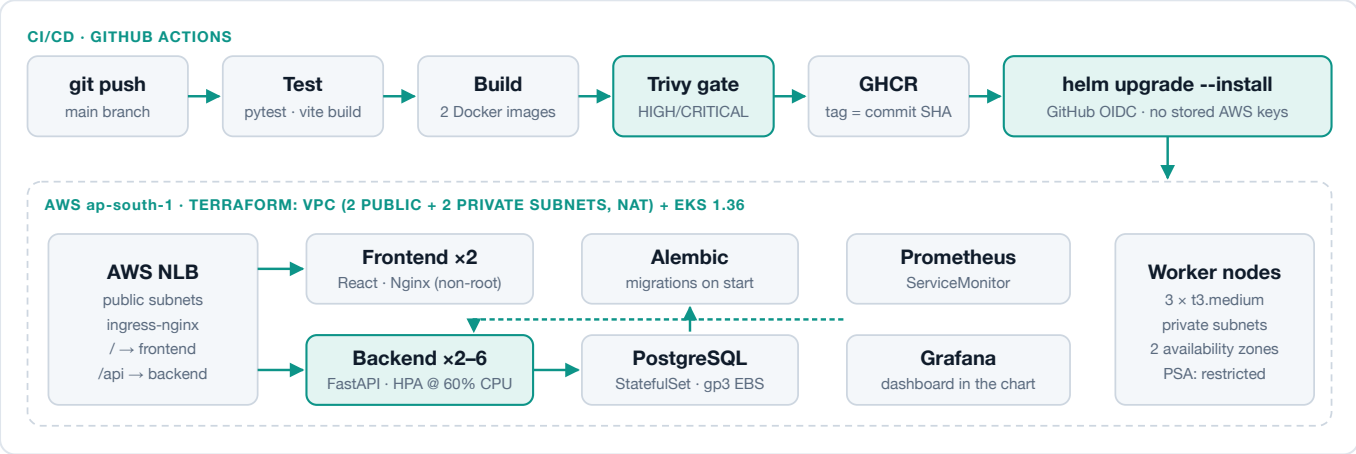
21
automated tests covering all 12 endpoints

0
fixable HIGH/CRITICAL CVEs in either image

64
AWS resources created by Terraform

~3 min
from git push to the new version live on EKS

Architecture



Technology stack

Frontend	React 18, Vite, plain CSS · served by unprivileged Nginx
Backend	FastAPI, SQLAlchemy 2, Pydantic 2, Alembic, Uvicorn
Database	PostgreSQL 16
Testing	pytest + FastAPI TestClient on in-memory SQLite
Containers	Docker multi-stage builds, Docker Compose
CI/CD	GitHub Actions, Trivy 0.74, GitHub Container Registry
Infrastructure	Terraform: AWS VPC, EKS, EBS CSI, IAM OIDC
Platform	Kubernetes 1.36, Helm, ingress-nginx, HPA, metrics-server
Observability	kube-prometheus-stack: Prometheus + Grafana

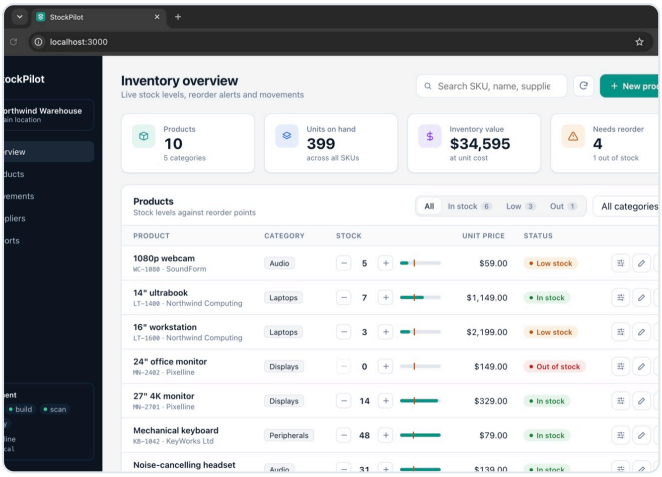
Repository layout

`backend/`	API, Alembic migrations, tests, Dockerfile
`frontend/`	React app, Nginx config, Dockerfile
`docker-compose.yml`	Full local stack in one command
`.github/workflows/`	CI/CD pipeline
`terraform/`	VPC, EKS, IAM (326 lines of HCL)
`helm/stockpilot/`	Kubernetes package for the whole app
`k8s/ · monitoring/`	Namespace, StorageClass, Prometheus values
`troubleshooting/`	Three deliberately broken manifests
`scripts/`	Cluster bootstrap, local cluster, load test

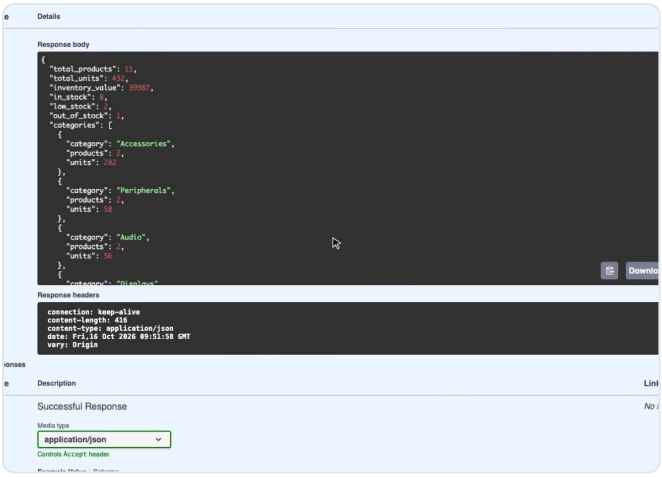
02 The application

M1 · Application

A React dashboard talks to a FastAPI backend through relative `/api` paths, so the browser never needs to know where the backend runs. Nginx (in Docker) or the Ingress (in Kubernetes) does the routing.



Dashboard: KPI cards, stock table with reorder bars, filters and quick ± buttons



Swagger UI generated by FastAPI: a live call to `/api/products/stats`

REST API

ENDPOINT	PURPOSE
`GET /health`	Liveness: the process is up (no DB call)
`GET /ready`	Readiness: database reachable, else 503
`GET /metrics`	Prometheus metrics
`GET /api/products`	List; search, status, category filters
`POST /api/products`	Create; 409 on duplicate SKU
`GET /api/products/{id}`	Read one product
`PUT /api/products/{id}`	Partial update; quantity edits audited
`DELETE /api/products/{id}`	Delete with its history
`POST …/{id}/adjust`	Stock movement; never below zero
`GET /api/products/stats`	KPIs and units per category
`GET /api/movements`	Audit trail, newest first

Data model

TABLE	KEY COLUMNS
`products`	sku (unique), name, category, quantity ≥ 0, reorder_level, unit_price, supplier
`stock_movements`	product_id (FK, cascade), delta, reason, note, quantity_after, created_at

Both tables are created by **Alembic migration 0001**, never by hand. The backend runs `alembic upgrade head` on start-up.

Frontend features

- KPI cards: products, units on hand, inventory value, items to reorder
- Status tabs, category filter and debounced search
- Create, edit and delete products; adjust stock with a reason and note
- Units-by-category chart and recent-movements feed
- Loading and backend-error states; responsive down to phone width

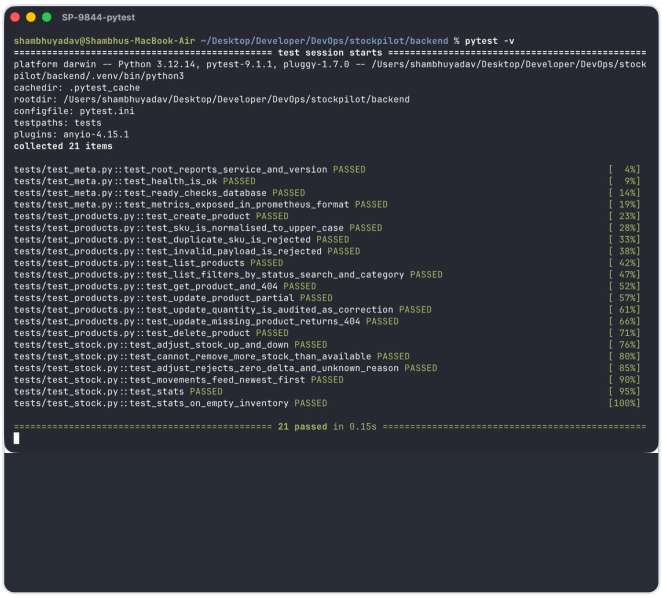
03 Testing, Git and Docker

M2 · M3 · M4

Automated tests

21 pytest tests exercise every endpoint, including the edge cases: duplicate SKUs, invalid payloads, missing products, and removing more stock than exists. `tests/conftest.py` gives each test a **fresh in-memory SQLite database** through FastAPI's dependency override, so the real PostgreSQL database is never touched.

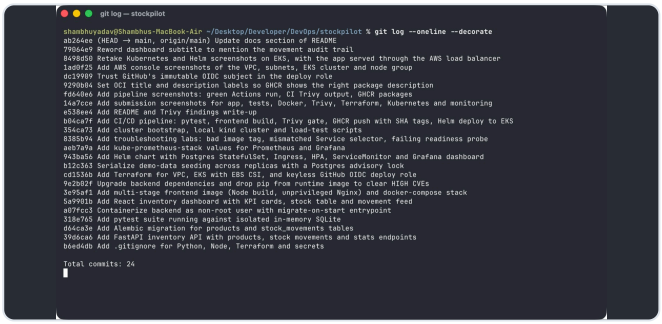
FILE	TESTS	COVERS
`test_meta.py`	4	root, /health, /ready, /metrics
`test_products.py`	11	CRUD, validation, filters, 404s
`test_stock.py`	6	adjustments, movements, stats



pytest -v: 21 passed in 0.3 s

Git and GitHub

- Public repository, **25 commits**, each describing one logical change
- `.gitignore` excludes `.env`, `__pycache__`, `node_modules`, `.venv`, Terraform state and `*.tfvars`
- No credentials anywhere in the history; the repository has **zero secrets**

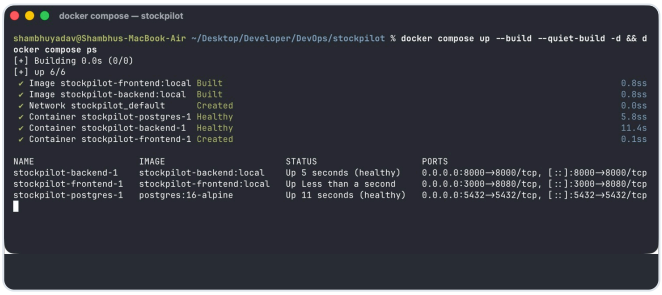


Commit history

Containers

	BACKEND	FRONTEND
Build	2 stages: venv, then slim runtime	2 stages: Node build, then Nginx
Runtime base	python:3.12-slim	nginx-unprivileged:1.27-alpine
User	UID 10001 (non-root)	UID 101 (non-root)
Port	8000	8080
Hardening	no pip in the image	`apk upgrade` at build

`docker compose up --build` starts PostgreSQL, then the backend once Postgres is healthy, then the frontend once the backend is healthy. The app opens at `localhost:3000`.



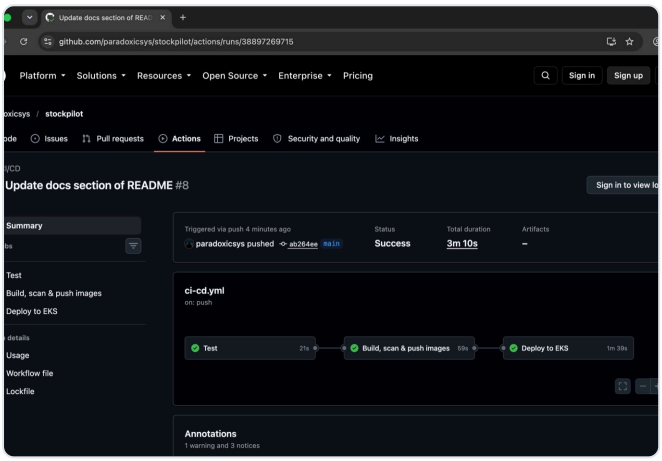
One command starts all three services

04 CI/CD and security scanning

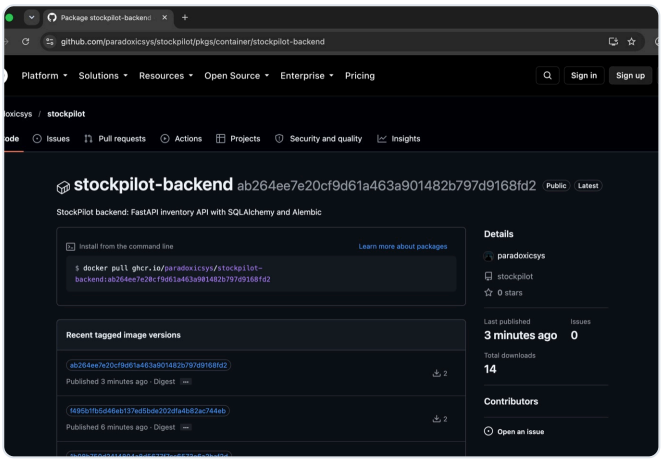
M5 · M6

One GitHub Actions workflow runs on every push and pull request to **main**. Each stage only starts if the previous one passed, so a broken build can never reach the registry or the cluster.

Test pytest · vite build › **Build** backend + frontend › **Trivy** fail on HIGH/CRITICAL › **Push** GHCR · :<commit SHA> › **Deploy** Helm on EKS · helm test



A green run: Test → Build, scan & push → Deploy to EKS



GHCR: each image is tagged with the full commit SHA, never latest

Pipeline design decisions

- **Tests first.** A failing test stops the run before any image is built.
- **Commit-SHA tags.** Every running pod traces back to an exact commit; the app's sidebar even shows the build it's running.
- **Keyless AWS access.** The deploy job exchanges a GitHub OIDC token for a short-lived AWS role, so the repository stores no AWS keys.
- **Safe rollout.** `helm upgrade --install --wait`, then `helm test`; on failure the job runs `helm rollback`.
- **Switchable deploy.** A repository variable turns deployment off after the cluster is destroyed, so the pipeline stays green.

Trivy: what it found

Trivy scans OS packages and Python libraries in both images. The **first scan failed**:

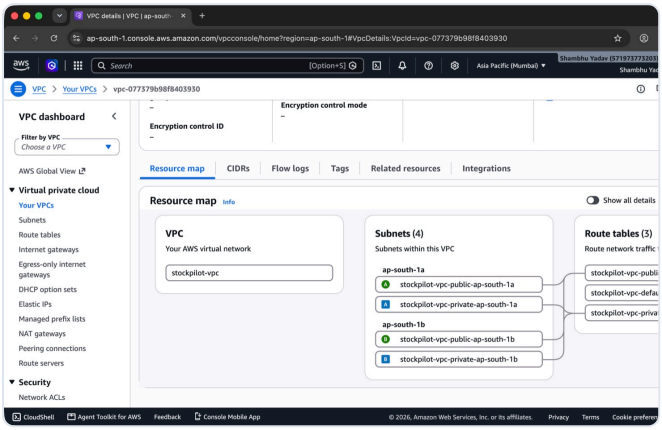
PACKAGE	ISSUE	FIX
`starlette 0.48`	CVE-2025-62727 (HIGH): Range-header DoS	1.7.0
`pip (vendored)`	urllib3, msgpack CVEs (HIGH)	pip removed
`setuptools 70`	CVE-2025-47273: path traversal	removed

> Upgrading the dependencies and building the backend in two stages, **with no pip in the runtime image**, brought both images to **0 fixable HIGH/CRITICAL**.

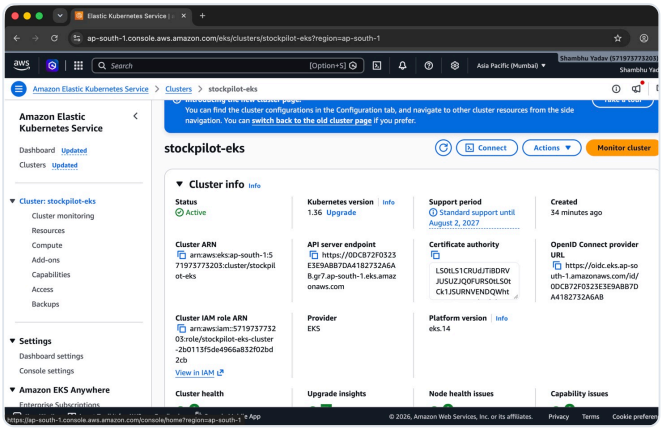
05 Infrastructure as Code on AWS

M7 · Terraform

Kubernetes manages workloads; it can't create the network or the cluster it runs on. Terraform describes that infrastructure as code. `terraform plan` shows 64 resources to create with 0 errors, and `terraform destroy` removes them all.



stockpilot-vpc: 4 subnets across 2 availability zones



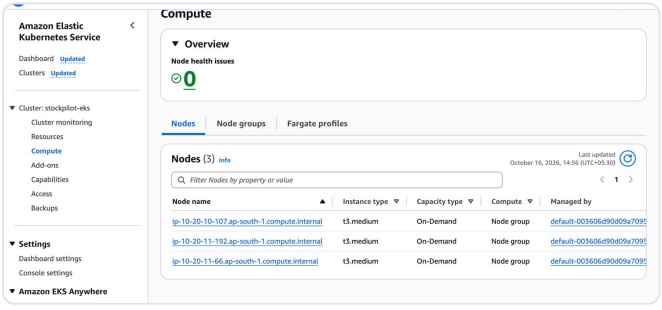
EKS 1.36: Active, in standard support until August 2027

What Terraform creates

VPC	10.20.0.0/16 in ap-south-1, DNS enabled
Public subnets ×2	Load balancer and NAT gateway; tagged for Kubernetes ELBs
Private subnets ×2	Worker nodes, no public IPs
NAT gateway	One, to keep cost down (one per AZ in production)
EKS cluster	Kubernetes 1.36; coredns, kube-proxy, vpc-cni, pod-identity add-ons
Node group	3 × t3.medium, scales 2–4, AL2023
EBS CSI driver	Persistent volumes for PostgreSQL, via Pod Identity
GitHub OIDC role	Lets only main-branch workflows of this repo deploy

Modules and practices

- Official `terraform-aws-modules` for VPC (v6) and EKS (v21), AWS provider v6, pinned in the lock file.
- `terraform.tfvars.example` is committed; the real `terraform.tfvars` and state are git-ignored.
- AWS credentials come from the CLI profile, never from files in the repo.
- The cluster creator gets admin through an EKS access entry, and so does the CI role.



Three t3.medium nodes in the managed node group

06 Kubernetes, Helm and autoscaling

M8 · Kubernetes + Helm

The whole application is one Helm chart. The pipeline installs it with the image tag of the commit being deployed; Helm keeps every revision, so a rollback is one command.

What the chart deploys

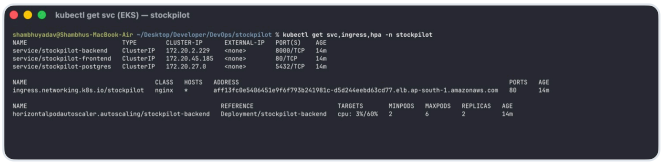
Backend	Deployment, 2–6 replicas (HPA), init container waits for Postgres
Frontend	Deployment, 2 replicas
PostgreSQL	StatefulSet with a 2 Gi encrypted gp3 volume
Services	ClusterIP for backend (8000), frontend (80), Postgres (5432)
Ingress	`/api`, `/docs` → backend; `/` → frontend
Secret	Random DB password on install, kept across upgrades
Monitoring	ServiceMonitor and Grafana dashboard ConfigMap
helm test	Pod that checks /ready and the API through both Services

Reliability and security

- **Liveness** uses `/health`, which never touches the database, so a DB outage doesn't restart every pod.
- **Readiness** uses `/ready`, which does check the DB, so unready pods leave the Service until they recover.
- Rolling updates with `maxUnavailable: 0`: no capacity drop during a deploy.
- All pods run as **non-root**, drop every Linux capability, use the RuntimeDefault seccomp profile, and pass the namespace's **restricted** Pod Security Standard.
- Replicas run migrations and seed data under a **Postgres advisory lock**, so they never race on start-up.



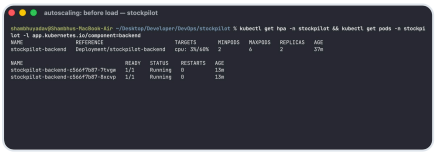
All pods Running on EKS nodes



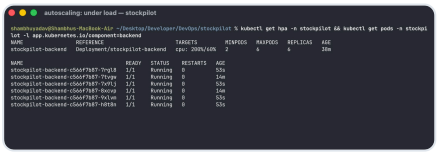
Services, Ingress on the AWS load balancer, and the HPA

Autoscaling under load

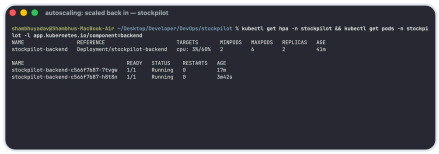
The HorizontalPodAutoscaler targets 60% of the backend's 100m CPU request. A 30-worker load test drove CPU to **200% of target**; the HPA scaled the backend from **2 to 6 pods** within a minute, then back to 2 once the traffic stopped.



Idle: 2 pods, 3% CPU



Under load: 200% / 60%, 6 pods

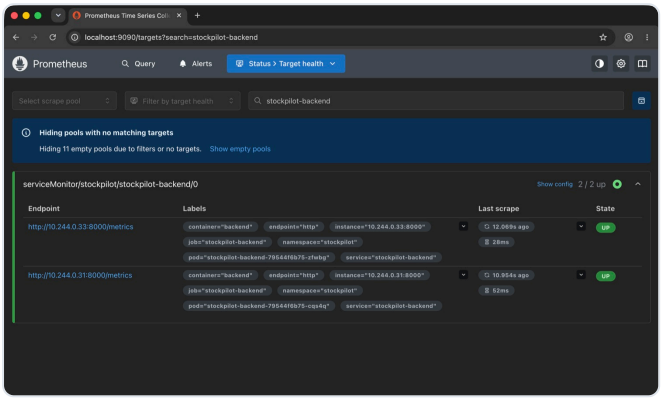


Load gone: back to 2 pods

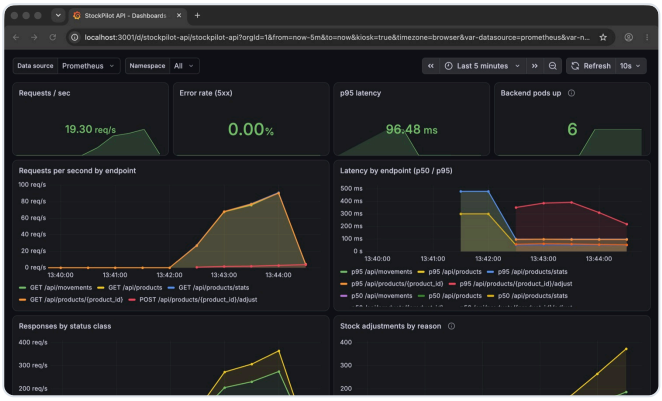
07 Observability and troubleshooting

M9 · Observability

The backend exposes Prometheus metrics at `/metrics`. kube-prometheus-stack discovers it through the chart's ServiceMonitor, and the Grafana dashboard is shipped in the same Helm chart, so monitoring arrives with every deploy.



Prometheus: every backend pod scraped and UP



Grafana "StockPilot API" dashboard with live data

Dashboard panels

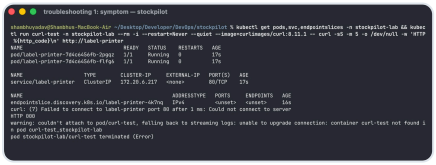
Requests / sec	overall and per endpoint
Error rate	share of 5xx responses
Latency	p50 and p95 per endpoint
Backend pods up	grows as the HPA scales out
CPU and memory	per backend pod
Business metric	`stockpilot_stock_adjustments_total` by reason

Troubleshooting labs

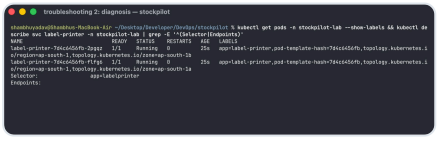
SYMPTOM	ROOT CAUSE
ImagePullBackOff	typo in the image tag
Service has no endpoints	selector doesn't match pod labels
Running but 0/1 Ready	readiness probe on a missing path

Each lab runs in its own namespace. The method: `get pods` → `describe` (events) → `logs` → `get endpointslices`.

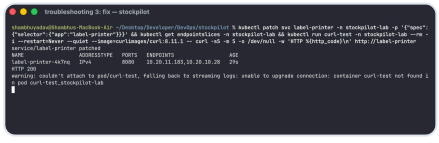
Lab walkthrough: a Service with no endpoints



1 · Pods Running, but endpoints empty and curl fails



2 · Selector `app=labelprinter` vs label `app=label-printer`



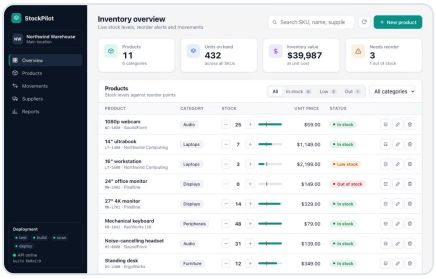
3 · Fixed selector: endpoints appear, HTTP 200

08 Live demo, lessons and rubric coverage

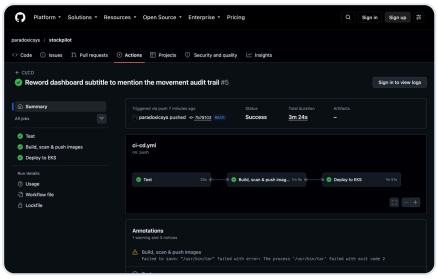
M10

Commit to production, end to end

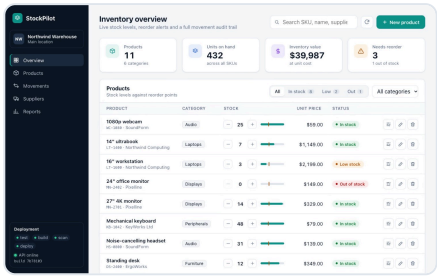
A one-line change to the dashboard subtitle was committed and pushed. The pipeline tested, built, scanned, pushed and deployed it to EKS with no manual step, in about **3½ minutes**.



Before: "…reorder alerts and movements"



Pipeline: all three jobs green



After: "…and a full movement audit trail"

Problems I hit and how I solved them

First security scan failed

Starlette and pip's bundled libraries had HIGH CVEs. Upgraded the dependencies and dropped pip from the runtime image: 0 fixable HIGH/CRITICAL.

Replicas raced on start-up

Two pods seeded demo data at once and one crashed. A Postgres advisory lock makes them take turns; verified with 4 replicas starting together.

Deploy job couldn't log in to AWS

GitHub now issues immutable OIDC subjects pinned to owner and repo IDs. Added that exact subject to the IAM trust policy in Terraform.

Persistent storage on EKS

Recent EKS needs the EBS CSI driver for volumes. Terraform installs it with a Pod Identity role, and an encrypted gp3 StorageClass is the default.

Rubric coverage

MODULE	EVIDENCE	STATUS
M1 Application	React + FastAPI + PostgreSQL, Alembic migration, 11 REST endpoints, responsive UI	✓
M2 Testing	21 tests on an isolated database; pytest.ini and conftest.py	✓
M3 Git	Public repository, 25 descriptive commits, .gitignore, no secrets	✓
M4 Docker	Multi-stage builds, non-root users, one-command Compose stack	✓
M5 CI/CD	On push to main: tests gate the build; images to GHCR tagged with the commit SHA	✓
M6 DevSecOps	Trivy gate on both images; a real CVE found, explained and fixed	✓
M7 Terraform	VPC with 2 public subnets + EKS with a node group in ap-south-1; tfvars.example	✓
M8 Kubernetes + Helm	Helm chart, 2+ replicas each, ClusterIP Services, Ingress, all pods Running	✓
M9 Observability	/metrics, Prometheus targets UP, Grafana panels with live data	✓
M10 Presentation	README, live commit → pipeline → EKS demo, recorded walkthrough video	✓